

1 Practice 1: GPIO Control of LED, Button, Relay and Buzzer

1.1 Objective

The objective of this practice is to use the GPIO pins of the ATmega328P to read a push button and control an LED, a relay, and a buzzer.

The button is configured as a digital input, while the LED, relay, and buzzer are configured as digital outputs.

1.2 Pin Configuration

The pins used in this practice are defined as follows:

```
const int BUTTON_PIN = 2;
const int LED_PIN = 13;
const int RELAY_PIN = 8;
const int BUZZER_PIN = 9;
```

The corresponding GPIO functions are:

Pin	Device	Function
D2	Button	Digital input
D13	LED	Digital output
D8	Relay	Digital output
D9	Buzzer	Digital output

1.3 GPIO Initialization

The button is configured as an input, while the LED, relay, and buzzer are configured as outputs.

```
pinMode(BUTTON_PIN, INPUT);
pinMode(LED_PIN, OUTPUT);
pinMode(RELAY_PIN, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);
```

The initial output states are set so that all devices are turned off:

```
digitalWrite(LED_PIN, LOW);
digitalWrite(RELAY_PIN, HIGH);
digitalWrite(BUZZER_PIN, LOW);
```

The LED and buzzer are active-high, while the relay is active-low.
Therefore:

Device	ON	OFF
<i>LED</i>	<i>HIGH</i>	<i>LOW</i>
<i>Relay</i>	<i>LOW</i>	<i>HIGH</i>
<i>Buzzer</i>	<i>HIGH</i>	<i>LOW</i>

1.4 Reading the Button

The state of the button is read using the digital input function:

```
int buttonState = digitalRead(BUTTON_PIN);
```

The button uses active-low logic. Therefore, a LOW signal indicates that the button is pressed:

Button State	Meaning
<i>LOW</i>	<i>Pressed</i>
<i>HIGH</i>	<i>Released</i>

The program checks whether the button state is LOW:

```
if (buttonState == LOW)
{
    ...
}
else
{
    ...
}
```

1.5 Controlling the LED, Relay and Buzzer

When the button is pressed, the LED and buzzer are turned on by outputting HIGH, while the relay is turned on by outputting LOW.

```
digitalWrite(LED_PIN, HIGH);
digitalWrite(RELAY_PIN, LOW);
digitalWrite(BUZZER_PIN, HIGH);
```

When the button is released, all devices are turned off:

```
digitalWrite(LED_PIN, LOW);
digitalWrite(RELAY_PIN, HIGH);
digitalWrite(BUZZER_PIN, LOW);
```

The complete control logic is:

$$\text{Button LOW (Pressed)} \rightarrow \begin{cases} \text{LED HIGH} & \rightarrow \text{LED ON} \\ \text{Relay LOW} & \rightarrow \text{Relay ON} \\ \text{Buzzer HIGH} & \rightarrow \text{Buzzer ON} \end{cases}$$

When the button is released:

$$\text{Button HIGH (Released)} \rightarrow \begin{cases} \text{LED LOW} & \rightarrow \text{LED OFF} \\ \text{Relay HIGH} & \rightarrow \text{Relay OFF} \\ \text{Buzzer LOW} & \rightarrow \text{Buzzer OFF} \end{cases}$$

1.6 Main Program

The program continuously reads the button state and updates the three output devices according to the button state.

```
void loop()
{
    int buttonState = digitalRead(BUTTON_PIN);

    if (buttonState == LOW)
    {
        digitalWrite(LED_PIN, HIGH);
        digitalWrite(RELAY_PIN, LOW);
        digitalWrite(BUZZER_PIN, HIGH);
    }
    else
    {
        digitalWrite(LED_PIN, LOW);
        digitalWrite(RELAY_PIN, HIGH);
        digitalWrite(BUZZER_PIN, LOW);
    }
}
```

1.7 Result

The ATmega328P successfully reads the digital state of the push button through a GPIO input and controls the LED, relay, and buzzer.

When the button is pressed, the LED is turned on, the relay is activated, and the buzzer is turned on.

When the button is released, the LED, relay, and buzzer are turned off.

This practice demonstrates the basic operation of GPIO input and output on the ATmega328P, including the use of active-low input and active-low output logic.

2 Practice 2: ADC and Sensor

2.1 Objective

The objective of this practice is to use the ATmega328 ADC to read an analog signal, collect 16 samples, calculate the average ADC value, convert the result into voltage, and send the result through serial communication.

2.2 ADC Configuration

The ADC is configured with the following parameters:

- ADC resolution: 10-bit
- ADC input channel: ADC2 (PC2)
- ADC reference voltage: $AVCC = 5\text{ V}$

- ADC prescaler: 128
- ADC conversion method: ADC interrupt
- Number of samples: 16

The ADC reference voltage is selected using the `REFS0` bit in the `ADMUX` register.

$$V_{\text{REF}} = V_{\text{CC}} = 5 \text{ V}$$

ADC2 is selected by setting the `MUX1` bit.

The ADC prescaler is set to 128 using the `ADPS2`, `ADPS1`, and `ADPS0` bits.

For a CPU clock of 16 MHz, the ADC clock is:

$$f_{\text{ADC}} = \frac{f_{\text{CPU}}}{128} = \frac{16 \text{ MHz}}{128} = 125 \text{ kHz}$$

2.3 ADC Initialization

The ADC is initialized by configuring the `ADMUX` and `ADCSRA` registers.

```
void ADC_Init() {
    ADMUX |= (1 << REFS0) | (1 << MUX1);

    ADCSRA = (1 << ADEN) | (1 << ADIE)
             | (1 << ADPS2) | (1 << ADPS1)
             | (1 << ADPS0);
}
```

The `ADEN` bit enables the ADC, while the `ADIE` bit enables the ADC conversion-complete interrupt.

2.4 ADC Interrupt

When an ADC conversion is completed, the ADC interrupt service routine is executed automatically.

```
ISR(ADC_vect) {
    adc_sum += ADC;
    sample_count++;

    if (sample_count >= 16) {
        adc_ready = true;
    } else {
        ADCSRA |= (1 << ADSC);
    }
}
```

The 10-bit ADC result is added to `adc_sum`. After 16 conversions, the program sets the `adc_ready` flag.

If fewer than 16 samples have been collected, the next ADC conversion is started by setting the `ADSC` bit.

2.5 Sampling and Averaging

The program collects 16 ADC samples before processing the data.

Let the ADC results be:

$$ADC_1, ADC_2, \dots, ADC_{16}$$

The average ADC value is calculated as:

$$ADC_{\text{avg}} = \frac{1}{16} \sum_{i=1}^{16} ADC_i$$

The program calculates the average using:

```
uint16_t avg_adc = adc_sum / 16;
```

Averaging multiple samples provides a more stable ADC value by reducing small variations between individual readings.

2.6 Conversion from ADC Value to Voltage

The ATmega328 has a 10-bit ADC, so the digital output ranges from 0 to 1023.

The input voltage is calculated using:

$$V_{\text{IN}} = \frac{ADC_{\text{avg}} \times V_{\text{REF}}}{1024}$$

With:

$$V_{\text{REF}} = 5 \text{ V}$$

the equation becomes:

$$V_{\text{IN}} = \frac{ADC_{\text{avg}} \times 5}{1024}$$

The program implements this conversion as:

```
float voltage = (avg_adc * 5.0) / 1024.0;
```

2.7 Serial Output

The averaged ADC value and the corresponding voltage are sent to the computer through UART at 9600 baud.

```
Serial.begin(9600);

Serial.print("Raw ADC: ");
Serial.print(avg_adc);
Serial.print(" | Voltage: ");
Serial.print(voltage);
Serial.println(" V");
```

The output has the following format:

```
Raw ADC: <ADC value> | Voltage: <voltage> V
```

2.8 Complete Program

```
#include <Arduino.h>
#include <avr/interrupt.h>

volatile uint16_t adc_sum = 0;
volatile uint8_t sample_count = 0;
volatile bool adc_ready = false;

void ADC_Init() {
    ADMUX |= (1 << REFS0) | (1 << MUX1);

    ADCSRA = (1 << ADEN) | (1 << ADIE)
             | (1 << ADPS2) | (1 << ADPS1)
             | (1 << ADPS0);
}

ISR(ADC_vect) {
    adc_sum += ADC;
    sample_count++;

    if (sample_count >= 16) {
        adc_ready = true;
    } else {
        ADCSRA |= (1 << ADSC);
    }
}

void setup() {
    Serial.begin(9600);
    ADC_Init();
}

void loop() {
    if (!adc_ready && sample_count == 0) {
        ADCSRA |= (1 << ADSC);
    }

    if (adc_ready) {
        uint16_t avg_adc = adc_sum / 16;

        float voltage = (avg_adc * 5.0) / 1024.0;

        Serial.print("Raw ADC: ");
        Serial.print(avg_adc);
        Serial.print(" | Voltage: ");
        Serial.print(voltage);
        Serial.println(" V");

        adc_sum = 0;
        sample_count = 0;
    }
}
```

```

        adc_ready = false;

        delay(500);
    }
}

```

3 Practice 3: PWM and Control Servo Motor

3.1 Objective

The objective of this practice is to generate PWM signals to control a servo motor and an LED using the ATmega328. The PWM signal of the servo motor is also observed using an oscilloscope.

3.2 Servo Motor PWM

The servo motor is connected to pin PB1, which corresponds to the OC1A output of Timer1.

Timer1 is configured in Fast PWM Mode 14. The prescaler is set to 8 and the value of ICR1 is used to determine the PWM period.

```

void Servo_PWM_Init(void) {
    DDRB |= (1 << SERVO_PIN);

    TCCR1A = (1 << COM1A1) | (1 << WGM11);

    TCCR1B = (1 << WGM13) |
              (1 << WGM12) |
              (1 << CS11);

    ICR1 = 39999;
}

```

With a CPU clock of 16 MHz and a prescaler of 8, the Timer1 clock is:

$$f_{\text{Timer1}} = \frac{16 \text{ MHz}}{8} = 2 \text{ MHz}$$

Therefore, one timer count corresponds to:

$$T_{\text{count}} = \frac{1}{2 \text{ MHz}} = 0.5 \mu\text{s}$$

The PWM period is:

$$T_{\text{PWM}} = (ICR1 + 1)T_{\text{count}}$$

With $ICR1 = 39999$:

$$T_{\text{PWM}} = 40000 \times 0.5 \mu\text{s} = 20 \text{ ms}$$

Therefore, the PWM frequency is:

$$f_{\text{PWM}} = \frac{1}{20 \text{ ms}} = 50 \text{ Hz}$$

Thus, Timer1 generates a PWM signal with a frequency of approximately 50 Hz for the servo motor.

3.3 Servo Angle Control

The servo angle is controlled by changing the pulse width of the PWM signal through the OCR1A register.

The program limits the angle to the range from 0 to 180 degrees.

```
void Servo_SetAngle(uint8_t angle) {
    if (angle > 180)
        angle = 180;

    uint16_t min_ocr = 999;
    uint16_t max_ocr = 4999;

    OCR1A = min_ocr +
        ((uint32_t)angle *
         (max_ocr - min_ocr)) / 180;
}
```

The pulse width is approximately mapped from 0.5 ms to 2.5 ms:

$$PW(\theta) = 0.5 \text{ ms} + \frac{\theta}{180}(2.5 - 0.5) \text{ ms}$$

where:

$$0^\circ \leq \theta \leq 180^\circ$$

Therefore:

$$\theta = 0^\circ \quad \Rightarrow \quad PW \approx 0.5 \text{ ms}$$

and:

$$\theta = 180^\circ \quad \Rightarrow \quad PW \approx 2.5 \text{ ms}$$

Changing the pulse width changes the position of the servo motor.

3.4 LED PWM

The LED is connected to PB3, which corresponds to the OC2A output of Timer2.

Timer2 is configured in Fast PWM mode with a prescaler of 64.

```
void LED_PWM_Init(void) {
    DDRB |= (1 << LED_PIN);

    TCCR2A = (1 << WGM20) |
        (1 << WGM21) |
        (1 << COM2A1);
}
```

```

    TCCR2B = (1 << CS22);
}

```

The LED brightness is controlled by changing the value of OCR2A. The servo angle is mapped to the 8-bit PWM range from 0 to 255.

```

void LED_SetBrightnessByAngle(uint8_t angle) {
    if (angle > 180)
        angle = 180;

    uint8_t pwm_val =
        (uint16_t)angle * 255 / 180;

    OCR2A = pwm_val;
}

```

The relationship between the servo angle and the LED PWM value is:

$$PWM_{LED} = \frac{\theta}{180} \times 255$$

Thus:

$$\theta = 0^\circ \Rightarrow PWM_{LED} = 0$$

and:

$$\theta = 180^\circ \Rightarrow PWM_{LED} = 255$$

Therefore, the LED brightness increases when the servo angle increases.

3.5 Servo and LED Operation

The program changes the servo angle from 0 to 180 degrees in steps of 5 degrees. At the same time, the LED PWM value is changed according to the servo angle.

After reaching 180 degrees, the servo angle is decreased back to 0 degrees.

```

for (uint8_t angle = 0; angle <= 180; angle += 5) {
    Servo_SetAngle(angle);
    LED_SetBrightnessByAngle(angle);

    _delay_ms(50);
}

for (int16_t angle = 180; angle >= 0; angle -= 5) {
    Servo_SetAngle((uint8_t)angle);
    LED_SetBrightnessByAngle((uint8_t)angle);

    _delay_ms(50);
}

```

The operating sequence is:

$$0^\circ \rightarrow 180^\circ \rightarrow 0^\circ$$

At the same time, the LED PWM changes from:

$$0 \rightarrow 255 \rightarrow 0$$

3.6 Oscilloscope Measurement

The PWM signal generated for the servo motor can be observed using an oscilloscope.

The oscilloscope is used to observe:

- PWM frequency
- PWM period
- Pulse width
- Change of pulse width when the servo angle changes

The expected PWM period is:

$$T_{\text{PWM}} \approx 20 \text{ ms}$$

and the expected frequency is:

$$f_{\text{PWM}} \approx 50 \text{ Hz}$$

The pulse width changes according to the servo angle and is expected to be approximately within the range:

$$0.5 \text{ ms} \leq PW \leq 2.5 \text{ ms}$$

When the servo angle changes, the oscilloscope should show that the pulse width changes while the PWM period remains approximately constant.

3.7 Result

Timer1 generates a PWM signal with a frequency of approximately 50 Hz to control the servo motor. The servo position is controlled by changing the PWM pulse width.

Timer2 generates PWM for the LED, and the LED brightness changes according to the servo angle.

The oscilloscope can be used to verify the PWM period, frequency, and pulse width of the servo control signal.

4 Practice 5: Timer and Scheduler

4.1 Objective

The objective of this practice is to use Timer0 to create a periodic time base and implement a simple scheduler for tasks with different execution periods.

The required tasks are:

- Read ADC every 100 ms.

- Update LED PWM every 20 ms.
- Read sensor every 100 ms.
- Send data through UART every 1000 ms.

In this program, Timer0 is used to generate a 1 ms time base. Software counters are then used to create the required task periods.

4.2 Timer0 Initialization

Timer0 is configured to generate a periodic interrupt. The timer uses CTC mode with a prescaler of 64.

```
void Timer0_Init(void) {
    TCCR0A = (1 << WGM01);

    TCCR0B = (1 << CS01) |
              (1 << CS00);

    OCROA = 249;

    TIMSK0 = (1 << OCIE0A);
}
```

With a CPU clock of 16 MHz and a prescaler of 64, the timer clock is:

$$f_{\text{Timer}} = \frac{16 \text{ MHz}}{64} = 250 \text{ kHz}$$

Therefore, each timer count has a duration of:

$$T_{\text{count}} = \frac{1}{250 \text{ kHz}} = 4 \mu\text{s}$$

With:

$$OCR0A = 249$$

the interrupt period is:

$$T_{\text{interrupt}} = (249 + 1) \times 4 \mu\text{s} = 1 \text{ ms}$$

Thus, Timer0 provides a 1 ms time base for the scheduler.

4.3 Timer Interrupt

The Timer0 Compare Match A interrupt sets a flag every 1 ms.

```
ISR(TIMERO_COMPA_vect) {
    timer_flag = 1;
}
```

The main loop checks this flag. When the flag is set, it is cleared and the software task counters are increased.

```

if (timer_flag) {
    timer_flag = 0;

    t_10ms++;
    t_20ms++;
    t_100ms++;
    t_1000ms++;
}

```

The 1 ms timer therefore acts as the basic clock for all scheduled tasks.

4.4 10 ms Task

The 10 ms counter is increased every 1 ms. When it reaches 10, the counter is reset.

```

if (t_10ms >= 10) {
    t_10ms = 0;

    // 10 ms task
}

```

Therefore, the task is executed every:

$$10 \times 1 \text{ ms} = 10 \text{ ms}$$

This provides a periodic task with a frequency of:

$$f_{10ms} = \frac{1}{10 \text{ ms}} = 100 \text{ Hz}$$

4.5 20 ms Task

The 20 ms task is used to update the LED PWM value according to the sensor value.

```

if (t_20ms >= 20) {
    t_20ms = 0;

    uint16_t temp_adc = adc_sensor_value;

    if (temp_adc < 300)
        temp_adc = 300;

    if (temp_adc > 555)
        temp_adc = 555;

    uint8_t pwm_out =
        (uint32_t)(temp_adc - 300) * 255 /
        (555 - 300);

    OCROA = pwm_out;
}

```

The ADC value is limited to the range:

$$300 \leq ADC \leq 555$$

The value is then mapped to the 8-bit PWM range:

$$0 \leq PWM \leq 255$$

The mapping equation is:

$$PWM = \frac{ADC - 300}{555 - 300} \times 255$$

The task is executed every:

$$20 \times 1 \text{ ms} = 20 \text{ ms}$$

Therefore, the LED PWM update frequency is:

$$f_{20ms} = \frac{1}{20 \text{ ms}} = 50 \text{ Hz}$$

4.6 100 ms Task

The sensor value is read from ADC channel 0 every 100 ms.

```
if (t_100ms >= 100) {  
    t_100ms = 0;  
  
    adc_sensor_value = ADC_Read(0);  
}
```

The ADC reading is performed using the following function:

```
uint16_t ADC_Read(uint8_t channel) {  
    ADMUX = (ADMUX & 0xF0) |  
            (channel & 0x0F);  
  
    ADCSRA |= (1 << ADSC);  
  
    while (ADCSRA & (1 << ADSC));  
  
    return ADC;  
}
```

The task period is:

$$100 \times 1 \text{ ms} = 100 \text{ ms}$$

Therefore, the sensor is read at a frequency of:

$$f_{100ms} = \frac{1}{100 \text{ ms}} = 10 \text{ Hz}$$

4.7 1000 ms Task

The UART task is executed every 1000 ms. It sends the current ADC value and PWM value to the computer.

```
if (t_1000ms >= 1000) {
    t_1000ms = 0;

    UART_SendString("Gia_tri_ADC_Cam_Bien:");
    UART_SendNumber adc_sensor_value;

    UART_SendString("|Duty_Cycle_PWM:");
    UART_SendNumber(OCR0A);

    UART_SendString("\r\n");
}
```

The task period is:

$$1000 \times 1 \text{ ms} = 1000 \text{ ms} = 1 \text{ s}$$

Therefore, UART data is transmitted once per second:

$$f_{1000\text{ms}} = \frac{1}{1 \text{ s}} = 1 \text{ Hz}$$

4.8 Scheduler Structure

The complete scheduler can be represented as a set of software counters driven by the 1 ms timer interrupt:

$$1 \text{ ms} \rightarrow \begin{cases} 10 \text{ ms} \rightarrow \text{Task 1} \\ 20 \text{ ms} \rightarrow \text{Task 2} \\ 100 \text{ ms} \rightarrow \text{Task 3} \\ 1000 \text{ ms} \rightarrow \text{Task 4} \end{cases}$$

The main loop checks each counter and executes the corresponding task when its period has elapsed.

4.9 Result

Timer0 provides a 1 ms time base for the scheduler. Software counters are used to generate different task periods without using a separate timer for each task.

The resulting task periods are:

Task	Period
Task 1	10 ms
Task 2	20 ms
Task 3	100 ms
Task 4	1000 ms

This scheduler allows tasks with different execution periods to run from a common 1 ms time base.

5 Practice 6: Control Step Motor

5.1 Objective

The objective of this practice is to control a stepper motor using the DRV8825 driver. The motor is controlled by generating pulse signals and setting the rotation direction. The motor is commanded to move through a sequence of predefined angles.

5.2 DRV8825 Connections

The control signals of the DRV8825 are connected to the ATmega328 as follows:

- PD2: EN (Enable)
- PD3: DIR (Direction)
- PD4: PUL (Pulse/Step)

The corresponding Arduino pins are defined as:

```
#define EN_PIN    2
#define DIR_PIN   3
#define PUL_PIN   4
```

The driver is enabled by setting the EN pin to LOW:

```
digitalWrite(EN_PIN, LOW);
```

5.3 Step and Angle Configuration

The motor is configured to use the following microstep setting:

$$M0 : M1 : M2 = 1 : 0 : 0$$

This configuration corresponds to half-step operation. In the tested configuration, 400 pulses are required for one complete revolution of 360 degrees.

Therefore, the number of steps per degree is:

$$StepsPerDegree = \frac{400}{360}$$

The program defines this relationship as:

```
const float STEPS_PER_DEGREE = 400.0 / 360.0;
```

For a required angular displacement $\Delta\theta$, the number of pulses is calculated by:

$$N = \Delta\theta \times \frac{400}{360}$$

where N is the required number of pulses.

5.4 Generating Step Pulses

The `stepMotor()` function generates the required number of pulses on the PUL pin.

```
void stepMotor(uint16_t steps, bool direction) {
    digitalWrite(DIR_PIN,
                 direction ? HIGH : LOW);

    for (uint16_t i = 0; i < steps; i++) {
        digitalWrite(PUL_PIN, HIGH);
        delayMicroseconds(1000);

        digitalWrite(PUL_PIN, LOW);
        delayMicroseconds(1000);
    }
}
```

The DIR pin determines the rotation direction. The PUL pin generates the step pulses. Each pulse consists of:

$$1 \text{ ms HIGH} + 1 \text{ ms LOW}$$

Therefore, the pulse period is approximately:

$$T_{\text{pulse}} = 2 \text{ ms}$$

and the corresponding pulse frequency is:

$$f_{\text{pulse}} = \frac{1}{2 \text{ ms}} = 500 \text{ Hz}$$

5.5 Moving to a Target Angle

The `moveToAngle()` function calculates the difference between the target angle and the current angle.

```
void moveToAngle(uint16_t target_angle) {
    int16_t angle_diff =
        target_angle - current_angle;

    if (angle_diff == 0)
        return;

    bool direction =
        (angle_diff > 0);

    uint16_t abs_angle_diff =
        abs(angle_diff);

    uint16_t steps_to_move =
        round(abs_angle_diff *
              STEPS_PER_DEGREE);

    stepMotor(steps_to_move, direction);
}
```

```

    current_angle = target_angle;
}

```

The direction is determined from the sign of the angle difference:

$$\Delta\theta = \theta_{\text{target}} - \theta_{\text{current}}$$

If:

$$\Delta\theta > 0$$

the motor rotates in one direction.

If:

$$\Delta\theta < 0$$

the motor rotates in the opposite direction.

The absolute angular difference is then converted into the required number of pulses.

5.6 Target Angle Sequence

The program defines the following target angles:

```

const uint16_t target_angles[] =
{
    0, 45, 90, 135, 180,
    225, 270, 315, 360
};

```

The motor therefore moves through the following sequence:

$$0^\circ \rightarrow 45^\circ \rightarrow 90^\circ \rightarrow 135^\circ \rightarrow 180^\circ$$

$$\rightarrow 225^\circ \rightarrow 270^\circ \rightarrow 315^\circ \rightarrow 360^\circ$$

At each target angle, the program waits for one second:

```

delay(1000);

```

The motor therefore remains at each target position before moving to the next position.

5.7 Returning to the Initial Position

After reaching the final target angle of 360 degrees, the program resets the software position to 0 degrees:

```

current_angle = 0;

```

The loop then starts again from the first target angle.

Therefore, the complete operating sequence is:

$$0^\circ \rightarrow 45^\circ \rightarrow 90^\circ \rightarrow 135^\circ \rightarrow 180^\circ$$

$$\rightarrow 225^\circ \rightarrow 270^\circ \rightarrow 315^\circ \rightarrow 360^\circ \rightarrow 0^\circ$$

5.8 Result

The ATmega328 controls the stepper motor through the DRV8825 driver using three control signals: Enable, Direction, and Pulse.

The motor position is controlled by the number of pulses generated on the PUL pin. The DIR signal determines the rotation direction, while the target angle is converted into the corresponding number of motor steps.

The tested program successfully moves the motor through the predefined angles from 0 to 360 degrees and repeats the sequence continuously.

6 Practice 7: Input Capture and Timer1

6.1 Objective

The objective of this practice is to use the Input Capture Unit of Timer1 to measure the pulse width and period of an input signal.

The input signal is connected to the ICP1 input pin, which corresponds to PB0 (D8) on the ATmega328.

6.2 Input Capture Pin

The ICP1 input is connected to PB0. The pin is configured as an input:

```
DDRB &= ~(1 << DDB0);
```

The Input Capture Unit uses the Timer1 counter to record the timer value when an input edge is detected.

6.3 Timer1 Configuration

Timer1 is configured for Input Capture operation. The timer uses a prescaler of 8.

```
void Timer1_ICP1_Init() {
    TCCR1A = 0;
    TCCR1B = 0;
    TCNT1 = 0;

    TCCR1B = (1 << ICNC1) |
             (1 << ICES1) |
             (1 << CS11);

    TIFR1 |= (1 << ICF1);
    TIMSK1 |= (1 << ICIE1);
}
```

With a CPU clock of 16 MHz and a prescaler of 8, the Timer1 clock is:

$$f_{\text{Timer1}} = \frac{16 \text{ MHz}}{8} = 2 \text{ MHz}$$

Therefore, one timer tick corresponds to:

$$T_{\text{tick}} = \frac{1}{2\text{MHz}} = 0.5\mu\text{s}$$

Thus, the Timer1 counter can be used to measure the time difference between two captured edges.

6.4 Input Capture Operation

The Input Capture Unit generates an interrupt whenever the configured input edge is detected.

The program initially detects the rising edge:

```
ICES1 = 1
```

When a rising edge is detected, the current Timer1 value is stored. The program then changes the capture configuration to detect the falling edge.

When the falling edge is detected, the Timer1 value is stored again. The difference between the two values represents the duration of the HIGH level of the input signal.

The capture process is:

Rising Edge → Falling Edge → Rising Edge → ...

6.5 Measuring Pulse Width

The Input Capture interrupt handles the rising and falling edges.

```
ISR(TIMER1_CAPT_vect) {
    uint16_t current_icr = ICR1;

    if (edge_state == 0) {
        t_rising = current_icr;

        if (t_last_rising != 0) {
            period_ticks =
                t_rising - t_last_rising;
        }

        t_last_rising = t_rising;

        TCCR1B &= ~(1 << ICES1);
        edge_state = 1;
    }
    else {
        t_falling = current_icr;

        pulse_width_ticks =
            t_falling - t_rising;

        TCCR1B |= (1 << ICES1);
        edge_state = 0;

        capture_complete = true;
    }
}
```

For a rising edge followed by a falling edge:

$$N_{\text{width}} = T_{\text{falling}} - T_{\text{rising}}$$

where N_{width} is the measured number of Timer1 ticks.

Since each tick corresponds to $0.5\ \mu\text{s}$:

$$T_{\text{HIGH}} = N_{\text{width}} \times 0.5 \mu\text{s}$$

6.6 Measuring Signal Period

The period is measured using two consecutive rising edges.

The program stores the previous rising-edge timestamp in `t_last_rising`.

The period in Timer1 ticks is calculated as:

$$N_{\text{period}} = T_{\text{rising,current}} - T_{\text{rising,previous}}$$

The corresponding period in microseconds is:

$$T_{\text{period}} = N_{\text{period}} \times 0.5 \mu\text{s}$$

The program performs this calculation with:

```
period_ticks = t_rising - t_last_rising;
```

6.7 Converting Timer Ticks to Time

After a complete measurement is captured, the measured values are converted from Timer1 ticks to microseconds.

```
uint32_t high_time_us = width_ticks / 2;
uint32_t period_us = period_t / 2;
```

Because one Timer1 tick corresponds to $0.5\ \mu\text{s}$:

$$T_{\text{HIGH}} = \frac{N_{\text{width}}}{2} \mu\text{s}$$

and:

$$T_{\text{period}} = \frac{N_{\text{period}}}{2} \mu\text{s}$$

6.8 UART Output

The measured pulse width and period are transmitted through USART at 9600 baud.

```
USART_SendString("Do_rong_xung_(HIGH):_");
USART_SendNum(high_time_us);

USART_SendString("_us_|_Chu_ky:_");
```

```
USART_SendNum(period_us);  
  
USART_SendString("_us\r\n");
```

The output therefore displays the measured HIGH pulse width and the signal period in microseconds.

6.9 Measurement Process

The complete measurement process can be summarized as:

Input Signal → ICP1 → Timer1 Capture → Calculate Time → UART

The rising edge is used to determine the beginning of the pulse and to calculate the period. The falling edge is used to determine the end of the HIGH pulse.

6.10 Result

Timer1 Input Capture is used to measure the timing characteristics of the input signal on ICP1.

With a prescaler of 8 and a 16 MHz CPU clock, each Timer1 tick corresponds to $0.5\mu\text{s}$. The program uses this time resolution to calculate the HIGH pulse width and signal period.

The measured values are displayed through the USART interface at 9600 baud.

7 Practice 8: I2C Communication

7.1 Objective

The objective of this practice is to understand the basic operation of the I2C communication protocol and implement the main I2C control functions using the hardware I2C module of the ATmega328.

The main I2C operations implemented in this practice are:

- I2C initialization
- START condition
- Data transmission
- Data reception with ACK
- Data reception with NACK
- STOP condition

There is no external I2C device connected in this practice. Therefore, the transmitted data is also displayed on LEDs for observation.

7.2 I2C Initialization

The ATmega328 hardware I2C peripheral is initialized using the following function:

```
void i2c_init(void) {
    TWSR = 0x00;
    TWBR = 72;
    TWCR = (1 << TWEN);
}
```

The TWEN bit enables the I2C hardware module.

The value of TWBR is used to configure the I2C clock frequency.

7.3 I2C START Condition

The START condition is generated using the following function:

```
void i2c_start(void) {
    TWCR = (1 << TWINT) |
           (1 << TWSTA) |
           (1 << TWEN);
}
```

The TWSTA bit requests the hardware to generate an I2C START condition.

The normal I2C communication sequence begins with a START condition:

START → Address → Data → STOP

7.4 I2C Data Transmission

The i2c_write() function sends one byte through the I2C interface.

```
void i2c_write(uint8_t data) {
    PORTB = (PORTB & 0xFC) |
            (data & 0x03);

    PORTD = (PORTD & 0x03) |
            (data & 0xFC);

    TWDR = data;

    TWCR = (1 << TWINT) |
           (1 << TWEN);
}
```

Before transmitting the byte through the I2C peripheral, the data is also mapped to the LED outputs.

The two least significant bits are displayed on PB0 and PB1:

$D_1D_0 \rightarrow PB1, PB0$

The remaining six bits are displayed on PD2 through PD7:

$D_7D_6D_5D_4D_3D_2 \rightarrow PD7, PD6, PD5, PD4, PD3, PD2$

Therefore, the complete 8-bit transmitted value can be observed using the LEDs.

7.5 I2C Address Transmission

The sensor address used in the test sequence is:

$0x68$

For an I2C write operation, the 7-bit address is shifted left by one bit and the write bit is added:

$$Address = (0x68 \ll 1) | 0$$

This value is transmitted using:

```
i2c_write((0x68 << 1) | 0);
```

The resulting I2C address byte contains both the sensor address and the write indication.

7.6 I2C Data Reception

Two functions are provided for receiving data.

The first function requests an ACK after receiving a byte:

```
uint8_t i2c_read_ack(void) {
    TWCR = (1 << TWINT) |
           (1 << TWEN) |
           (1 << TWEA);

    return TWDR;
}
```

The second function receives a byte without sending an ACK:

```
uint8_t i2c_read_nack(void) {
    TWCR = (1 << TWINT) |
           (1 << TWEN);

    return TWDR;
}
```

The ACK and NACK functions are included to implement the two basic types of I2C read operation.

In this practice, there is no external I2C device providing data, so these functions are not used in the main test sequence.

7.7 I2C STOP Condition

The communication is terminated using the STOP condition:

```
void i2c_stop(void) {
    TWCR = (1 << TWINT) |
           (1 << TWSTO) |
           (1 << TWEN);
}
```

The TWST0 bit requests the hardware to generate an I2C STOP condition. Therefore, a complete transmission has the following structure:

START → I2C Address → Data → STOP

7.8 Test Data Sequence

The program uses a sequence of bytes representing typical register addresses and configuration data of an I2C sensor:

```
uint8_t sample_data[] = {  
    0x75,  
    0x6B,  
    0x00,  
    0x3B,  
    0x1C,  
    0x08,  
    0x43,  
    0x1A  
};
```

These values represent example commands such as:

- 0x75: WHO_AM_I register
- 0x6B: PWR_MGMT_1 register
- 0x00: Wake-up configuration data
- 0x3B: Accelerometer data register
- 0x1C: Accelerometer configuration register
- 0x08: Accelerometer range configuration
- 0x43: Gyroscope data register
- 0x1A: Configuration register

The program sends one byte from this sequence during each iteration.

7.9 Communication Sequence

The main loop performs the following operations:

```
while (1) {  
    i2c_start();  
  
    i2c_write((0x68 << 1) | 0);  
  
    i2c_write(sample_data[current_index]);  
  
    i2c_stop();  
}
```

```

current_index =
    (current_index + 1) % total_patterns;

_delay_ms(500);
}

```

The resulting communication sequence is:

START → Address + Write → Register/Data → STOP

A new byte from the test sequence is transmitted every 500 ms.

7.10 LED Data Display

The transmitted byte is simultaneously displayed on the LEDs connected to PB0, PB1, and PD2 through PD7.

Therefore, the 8-bit data can be visually observed as:

$$D_7 D_6 D_5 D_4 D_3 D_2 D_1 D_0$$

with the corresponding LED mapping:

Data bit	LED output
D_0	$PB0$
D_1	$PB1$
D_2	$PD2$
D_3	$PD3$
D_4	$PD4$
D_5	$PD5$
D_6	$PD6$
D_7	$PD7$

This allows the transmitted byte to be observed without requiring an external I2C device.

7.11 Result

The practice implements the basic I2C operations using the ATmega328 hardware I2C peripheral.

The program generates START and STOP conditions, sends the I2C address and data bytes, and provides functions for receiving data with ACK or NACK.

Because no external I2C device is connected, the transmitted bytes are displayed on LEDs for observation. The test sequence sends one byte every 500 ms.

8 Practice 9: Serial Control of a Servo Motor Using ATmega328

8.1 Objective

The objective of this practice is to control a servo motor through serial communication with a computer.

The program receives an angle command through the serial interface, checks whether the command is valid, and moves the servo to the requested angle if the value is within the permitted range.

The valid servo angle range is:

$$0^\circ \leq \theta \leq 180^\circ$$

8.2 Servo Configuration

The servo motor is connected to digital pin 9.

```
const int SERVO_PIN = 9;
const int MIN_ANGLE = 0;
const int MAX_ANGLE = 180;
```

The servo is initialized in the `setup()` function:

```
Serial.begin(9600);

servo.attach(SERVO_PIN);
servo.write(90);
```

The serial communication is configured at:

$$\text{Baudrate} = 9600 \text{ baud}$$

The servo is initially positioned at:

$$\theta = 90^\circ$$

8.3 Receiving Serial Commands

The program continuously checks whether data is available through the serial interface.

```
while (Serial.available() > 0)
{
    char receivedChar = Serial.read();

    if (receivedChar == '\n' ||
        receivedChar == '\r')
    {
        if (inputBuffer.length() > 0)
        {
            processCommand(inputBuffer);
            inputBuffer = "";
        }
    }
    else
    {
        inputBuffer += receivedChar;
    }
}
```

The received characters are stored in `inputBuffer`. When a newline or carriage return is received, the accumulated string is processed as one command.

8.4 Command Validation

Before converting the command into an angle, the program checks whether the input contains a valid numerical value.

```
bool validNumber = true;

for (unsigned int i = 0;
     i < command.length(); i++)
{
    if (!isDigit(command[i]) &&
        !(i == 0 && command[i] == '-'))
    {
        validNumber = false;
        break;
    }
}
```

If the command contains an invalid character, the program rejects the command and sends an error message:

```
if (!validNumber)
{
    Serial.print("\nERROR: Invalid command.");
    return;
}
```

8.5 Checking the Servo Angle

After the command has been verified as a number, it is converted into an integer angle:

```
int angle = command.toInt();
```

The program then checks whether the angle is within the permitted range:

$$0^\circ \leq \theta \leq 180^\circ$$

```
if (angle < MIN_ANGLE ||
    angle > MAX_ANGLE)
{
    Serial.print(
        "\nERROR: Angle must be between "
        "0 and 180 degrees.\n"
    );
    return;
}
```

If the angle is outside this range, the servo is not moved and an error message is sent to the computer.

8.6 Controlling the Servo

If the command is valid and the angle is within the permitted range, the servo is moved to the requested position:

```
servo.write(angle);
```

The program then sends a confirmation message:

```
Serial.print("\nOK: Servo moved to ");  
Serial.print(angle);  
Serial.println(" degrees.");
```

Therefore, a valid command follows the sequence:

Serial Command → Validate → Check Range → Move Servo → Confirmation

An invalid command follows:

Serial Command → Validate → Reject → Error Message

8.7 Example of Operation

For a valid command such as:

```
90
```

the program moves the servo to 90 degrees and sends a confirmation message.

For a command outside the permitted range, such as:

```
200
```

the program rejects the command and sends an error message indicating that the angle must be between 0 and 180 degrees.

For a command containing invalid characters, the program reports:

```
ERROR: Invalid command.
```

8.8 Result

The ATmega328 receives servo angle commands from a computer through the serial interface at 9600 baud.

The program validates the received command before controlling the servo. Valid angles from 0 to 180 degrees are accepted and used to control the servo, while invalid commands or out-of-range angles are rejected with an error message.

After each valid command, the program sends a confirmation containing the commanded servo angle.

9 Practice 10: Motion Detection Using PIR Sensor

9.1 Objective

The objective of this practice is to detect motion using a PIR sensor connected to the ATmega328.

When the PIR sensor detects motion, the LED is turned on and a status message is sent through UART. When motion is no longer detected, the LED is turned off and the corresponding status message is sent.

The program also prevents repeated UART messages when the PIR state does not change.

9.2 Hardware Configuration

The PIR sensor is connected to digital pin 2 and the LED is connected to digital pin 13.

```
const int PIR_PIN = 2;
const int LED_PIN = 13;
```

The PIR pin is configured as an input and the LED pin is configured as an output:

```
pinMode(PIR_PIN, INPUT);
pinMode(LED_PIN, OUTPUT);
```

The serial interface is configured at:

Baudrate = 9600 baud

9.3 Reading the PIR Sensor

The PIR sensor state is read using `digitalRead()`:

```
int currentState = digitalRead(PIR_PIN);
```

The sensor is configured as active-low. Therefore, a LOW signal indicates motion detection in this implementation.

The states are interpreted as:

$$PIR = \begin{cases} LOW & \text{Motion detected} \\ HIGH & \text{No motion} \end{cases}$$

9.4 Detecting a State Change

The program stores the previous PIR state in `lastState`:

```
int lastState = LOW;
```

The current state is compared with the previous state:

```
if (currentState != lastState)
{
    ...
}
```

Therefore, the motion status is processed only when the PIR state changes.

This prevents the program from repeatedly sending the same UART message while the sensor remains in the same state.

9.5 Motion Detected

When the PIR output changes to LOW, the program determines that motion has been detected.

```
if (currentState == LOW)
{
    digitalWrite(LED_PIN, HIGH);

    Serial.println("Motion_detected.");
}
```

The LED is turned on and the following message is sent through UART:

```
Motion detected.
```

9.6 No Motion

When the PIR output changes to HIGH, the program determines that motion is no longer detected.

```
else
{
    digitalWrite(LED_PIN, LOW);

    Serial.println("No_motion.");
}
```

The LED is turned off and the following message is sent through UART:

```
No motion.
```

9.7 Updating the Previous State

After processing the state change, the current PIR state is stored as the previous state:

```
lastState = currentState;
```

The next loop iteration then compares the new sensor state with this stored value. The complete state-change process is:

Read PIR → Compare with Previous State → State Changed?

If the state has changed:

Process State → Update LED → Send UART Message

If the state has not changed, no new message is transmitted.

9.8 Result

The ATmega328 successfully reads the digital output of the PIR sensor and determines the motion state.

When motion is detected, the LED is turned on and the message “Motion detected.” is sent through UART. When there is no motion, the LED is turned off and the message “No motion.” is sent.

The program only sends a new status message when the PIR state changes, preventing repeated transmission of the same status.

10 Practice 11: LDR and Light Detection Using ATmega328

10.1 Objective

The objective of this practice is to measure light intensity using an LDR connected to the ADC of the ATmega328.

The ADC value is used for three purposes:

- Determine whether the environment is dark or bright.
- Control an LED according to the light condition.
- Display the ADC value on a 4-digit 7-segment display.

The ADC value and light condition are also transmitted through UART.

10.2 Hardware Configuration

The LDR is connected to analog input A0:

```
const int LIGHT_PIN = A0;
```

The LED is connected to digital pin 13:

```
const int LED_PIN = 13;
```

The 4-digit 7-segment display uses the following segment pins:

```
const int SEGMENT_PINS[8] = {  
    3, 4, 5, 6, 7, 8, 9, 10  
};
```

The four digit-selection pins are:

```
const int DIGIT_PINS[4] = {  
    11, 12, A1, A2  
};
```

The 7-segment display uses active-low logic. Therefore:

$$LOW = ON$$

and:

$$HIGH = OFF$$

10.3 Reading the LDR

The analog value from the LDR is continuously read using the ATmega328 ADC:

```
int adcValue = analogRead(LIGHT_PIN);
```

The ADC value is represented by:

$$0 \leq ADC \leq 1023$$

The current ADC value is used directly for light-condition detection, UART transmission, and display.

10.4 Dark and Bright Detection

A threshold value is defined to distinguish between dark and bright conditions:

```
const int DARK_THRESHOLD = 400;
```

The light condition is determined using:

$$Condition = \begin{cases} \text{Dark,} & ADC < 400 \\ \text{Bright,} & ADC \geq 400 \end{cases}$$

The program controls the LED according to this condition:

```
if (adcValue < DARK_THRESHOLD)
{
    digitalWrite(LED_PIN, HIGH);
}
else
{
    digitalWrite(LED_PIN, LOW);
}
```

Therefore, the LED is turned on when the measured ADC value is below the dark threshold and turned off when the value is equal to or above the threshold.

10.5 Displaying the ADC Value

The ADC value is displayed using a 4-digit 7-segment display.

The program separates the ADC value into four decimal digits:

```
digits[0] = (number / 1000) % 10;
digits[1] = (number / 100) % 10;
digits[2] = (number / 10) % 10;
digits[3] = number % 10;
```

For example, if:

$$ADC = 523$$

the four digits are:

$$0, 5, 2, 3$$

The display therefore represents:

0523

10.6 7-Segment Digit Patterns

The program stores the segment patterns for the digits from 0 to 9:

```
const byte DIGIT_PATTERN[10] = {
    0b00111111,
    0b00000110,
    0b01011011,
    0b01001111,
    0b01100110,
    0b01101101,
    0b01111101,
    0b00000111,
    0b01111111,
    0b01101111
};
```

The `setSegments()` function converts a digit pattern into the corresponding segment outputs.

```
void setSegments(byte pattern)
{
    for (int i = 0; i < 8; i++)
    {
        if (bitRead(pattern, i))
        {
            digitalWrite(SEGMENT_PINS[i], LOW);
        }
        else
        {
            digitalWrite(SEGMENT_PINS[i], HIGH);
        }
    }
}
```

Because the display is active-low, a LOW output turns a segment on.

10.7 4-Digit Multiplexing

The four digits are controlled using multiplexing. Only one digit is activated at a time, while the program rapidly switches between the four digits.

The `displayNumber()` function performs this operation:

```
for (int i = 0; i < 4; i++)
{
    turnOffDigits();

    setSegments(DIGIT_PATTERN[digits[i]]);
}
```

```

    digitalWrite(DIGIT_PINS[i], LOW);

    delay(2);

    digitalWrite(DIGIT_PINS[i], HIGH);
}

```

The sequence is:

Turn OFF all digits → Set segments → Turn ON one digit → Short delay → Turn OFF digit

This process is repeated continuously to maintain the appearance of a four-digit display.

10.8 Updating the Display Value

The ADC is read continuously, but the value used for the 7-segment display is updated every 100 ms.

```

static int displayValue = 0;
static unsigned long lastDisplayUpdate = 0;

if (millis() - lastDisplayUpdate >= 100)
{
    lastDisplayUpdate = millis();

    displayValue = adcValue;
}

```

Therefore, the ADC measurement and the display update operate at different rates:

ADC → Continuous Reading

while:

7-Segment Value → Update Every 100 ms

10.9 UART Output

The program sends the ADC value and the corresponding light condition through UART.

The UART interface is configured at:

Baudrate = 9600 baud

The output is updated every 100 ms:

```

if (millis() - lastSerialUpdate >= 100)
{
    lastSerialUpdate = millis();

    Serial.print("ADC:");
    Serial.print(adcValue);
}

```

```

    if (adcValue < DARK_THRESHOLD)
    {
        Serial.println("_Dark");
    }
    else
    {
        Serial.println("_Bright");
    }
}

```

The transmitted information has the following form:

```
ADC: 350 - Dark
```

or:

```
ADC: 700 - Bright
```

depending on the measured light level.

10.10 Overall Operation

The overall operation of the program can be summarized as:

LDR → ADC → ADC Value

The ADC value is then used by three independent functions:

$$\text{ADC Value} \rightarrow \begin{cases} \text{Dark/Bright Detection} \\ \text{LED Control} \\ \text{7-Segment Display} \\ \text{UART Output} \end{cases}$$

The 7-segment display shows the latest ADC value, while the LED indicates the detected light condition.

10.11 Result

The ATmega328 continuously reads the analog signal from the LDR using its ADC.

When the ADC value is below 400, the environment is classified as dark and the LED is turned on. When the ADC value is 400 or higher, the environment is classified as bright and the LED is turned off.

The ADC value is displayed on a 4-digit active-low 7-segment display using multiplexing. The displayed value is updated every 100 ms.

The ADC value and the corresponding light condition are also transmitted through UART at 9600 baud.